

OptiCrop – Smart Agricultural Production Optimization Engine

Project Description:

The project “Smart Agricultural Production Optimization Engine” aims to develop an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil temperature, humidity, pH, rainfall, and crop types, OptiCrop seeks to provide intelligent recommendations to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the OptiCrop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the development of evidence-based agricultural strategies. The OptiCrop project revolutionizes agricultural practices by providing farmers with a smart production optimization engine. By integrating key environmental factors and crop-specific recommendations, OptiCrop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable agricultural practices.

Scenario 1: Smart Crop Recommendation for Farmers

A farmer enters soil and environmental details such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

Scenario 2: Crop Suitability and Environmental Assessment

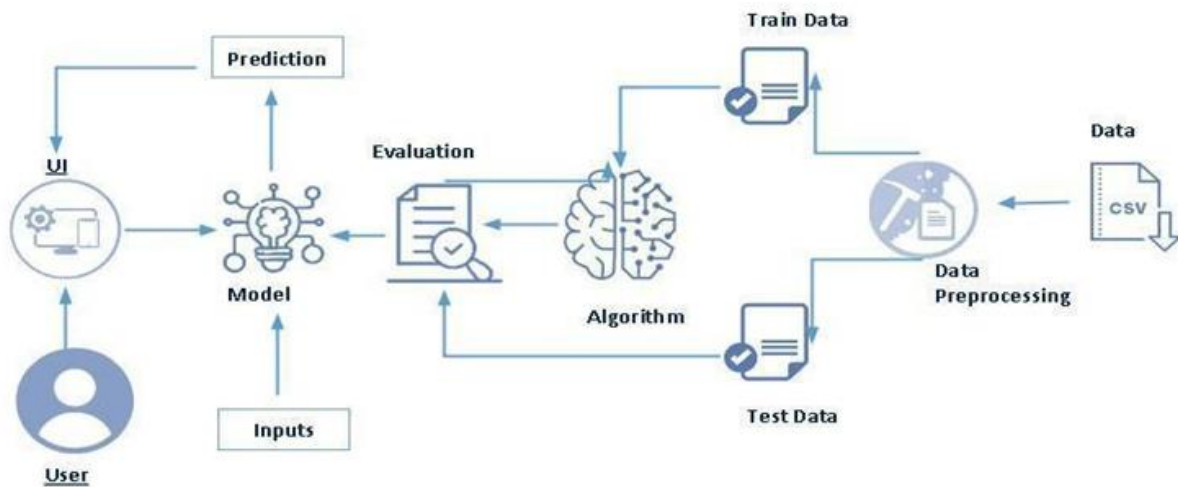
A user wants to understand whether the current soil and climate conditions are suitable for a particular crop. The application evaluates the input parameters and provides insights about crop compatibility, environmental suitability, and productivity potential.

Scenario 3: Agricultural Research and Policy Planning

An agricultural researcher or policymaker uses the system to analyze crop-environment relationships and identify patterns in agricultural production. The platform helps in making data-driven decisions for sustainable farming strategies and resource optimization.

Technical Architecture:

Description: This diagram illustrates the workflow of the crop recommendation system. The dataset is first preprocessed to clean and prepare the data. It is then divided into training and testing datasets. The training data is used to train the machine learning algorithm, while the testing data is used to evaluate the model's performance. Once the model is trained, users provide input through the user interface (UI), and the model processes the input to generate accurate crop predictions.



Pre-requisites:

The project was developed using a set of powerful Python-based tools and libraries for data processing, machine learning, visualization, and deployment. These technologies provide the foundation for efficient model development, analysis, and application deployment.

Anaconda Navigator: A desktop graphical user interface used to launch applications and manage Python packages and environments for data science. <https://www.anaconda.com/download>

PyCharm: An Integrated Development Environment (IDE) specifically designed for Python programming, offering code analysis and debugging tools. <https://www.jetbrains.com/pycharm/>

NumPy: Core library for numerical computing in Python, providing support for large, multi-dimensional arrays and mathematical functions. <https://numpy.org/doc/stable/>

Pandas: Data manipulation and analysis library offering data structures and operations for manipulating numerical tables and time series. <https://pandas.pydata.org/docs/>

Scikit-learn: Machine learning library featuring classification, regression, and clustering algorithms like support vector machines and random forests. <https://scikit-learn.org/stable/>

Matplotlib: Comprehensive library for creating static, animated, and interactive visualizations in Python. <https://matplotlib.org/stable/>

Seaborn: Data visualization library based on matplotlib that provides a high-level interface for drawing attractive statistical graphics. <https://seaborn.pydata.org/>

Flask: A lightweight WSGI web application framework used to build web APIs and deploy machine learning models. <https://flask.palletsprojects.com/>

Project Workflow:

Epic 1: Define Problem and Understanding

Story 1: Identify and define the agricultural problem that the crop recommendation system aims to solve.

Story 2: Gather and analyze business requirements to understand project objectives and expected outcomes.

Story 3: Conduct a literature survey to study existing crop recommendation techniques and machine learning approaches.

Story 4: Analyze the social and business impact of implementing an AI-driven crop recommendation solution.

Epic 2: Data Collection and Analysis

Story 1: Download and collect the agricultural dataset from reliable sources.

Story 2: Import the required Python libraries for data analysis, visualization, and machine learning.

Story 3: Read and explore the dataset to understand its structure, features, and target variables.

Story 4: Perform univariate analysis to examine the distribution and characteristics of individual features.

Story 5: Conduct bivariate analysis to identify relationships between agricultural parameters and crop suitability.

Story 6: Perform multivariate analysis to discover patterns and interactions among multiple variables.

Epic 3: Data Pre-Processing

Story 1: Check for null values and handle missing data to improve dataset quality.

Story 2: Detect and treat outliers that may affect model performance.

Story 3: Extract seasonal crop information and prepare relevant features for analysis.

Story 4: Split the dataset into training and testing sets for model development and evaluation.

Epic 4: Model Building

Story 1: Apply K-Means Clustering to identify patterns and group similar agricultural conditions.

Story 2: Train a Logistic Regression model to predict suitable crops based on environmental parameters.

Story 3: Evaluate model performance using appropriate metrics and compare results.

Story 4: Save the best-performing model for deployment and generate crop predictions based on user-provided inputs.

Epic 5: Application Building

Story 1: Design and develop HTML pages to create an interactive and user-friendly interface.

Story 2: Build the Python backend code and integrate it with the trained machine learning model.

Story 3: Run, test, and validate the application to ensure accurate crop recommendations and smooth functionality.

Epic 1: Define Problem and Understanding

Story 1: Identify and define the agricultural problem that the crop recommendation system aims to solve.

This step defines the core issue in traditional farming, where crop selection is mostly based on experience instead of data. The problem is identified as the lack of a scientific system that considers soil nutrients (N, P, K) and environmental factors for crop selection. OptiCrop is defined as a solution that uses these parameters to recommend suitable crops using machine learning.

Story 2: Gather and analyze business requirements to understand project objectives and expected outcomes.

This step identifies what the system must achieve. The application should take soil and climate inputs, process them using a trained ML model, and output crop recommendations. The main requirement is to provide accurate, fast, and user-friendly predictions for farmers and researchers.

Story 3: Conduct a literature survey to study existing crop recommendation techniques and machine learning approaches.

Existing crop recommendation systems and ML approaches are studied using research papers and datasets. Algorithms like KNN, Decision Tree, and Random Forest are compared to understand their effectiveness. This helps in selecting a suitable model and improving prediction accuracy.

Story 4: Analyze the social and business impact of implementing an AI-driven crop recommendation solution.

The system helps farmers improve yield and reduce resource waste by giving data-based crop suggestions. It also supports researchers and policymakers in making better agricultural decisions. Overall, it promotes efficient and sustainable farming practices.

Epic 2: Data Collection and Analysis

Story 1: Download and collect the agricultural dataset from reliable sources.

The dataset is collected from a reliable source such as Kaggle or an agricultural research repository. It contains soil and environmental parameters along with crop labels used for training the model. The dataset is downloaded and stored in the project directory for further processing.

```
1 N,P,K,temperature,humidity,ph,rainfall,label
2 90,42,43,20.87974371,82.00274423,6.502985292000001,202.9355362,rice
3 85,58,41,21.77046169,80.31964408,7.038096361,226.6555374,rice
4 60,55,44,23.00445915,82.3207629,7.840207144,263.9642476,rice
5 74,35,40,26.49109635,80.15836264,6.980400905,242.8640342,rice
6 78,42,42,20.13017482,81.60487287,7.628472891,262.7173405,rice
7 69,37,42,23.05804872,83.37011772,7.073453503,251.0549998,rice
8 69,55,38,22.70883798,82.63941394,5.70080568,271.3248604,rice
9 94,53,40,20.27774362,82.89408619,5.718627177999999,241.9741949,rice
10 89,54,38,24.51588066,83.53521629999999,6.685346424,230.4462359,rice
11 68,58,38,23.22397386,83.03322691,6.336253525,221.2091958,rice
12 91,53,40,26.52723513,81.41753846,5.386167788,264.6148697,rice
13 90,46,42,23.97898217,81.45061596,7.50283396,250.0832336,rice
14 78,58,44,26.80079604,80.88684822,5.108681786,284.4364567,rice
15 93,56,36,24.01497622,82.05687182,6.98435366,185.2773389,rice
16 94,50,37,25.66585205,80.66385045,6.94801983,209.5869708,rice
17 60,48,39,24.28209415,80.30025587,7.042299068999999,231.0863347,rice
18 85,38,41,21.58711777,82.7883708,6.2490506560000005,276.65524589999995,rice
19 91,35,39,23.79391957,80.41817957,6.970859754,206.2611855,rice
20 77,38,36,21.8652524,80.1923008,5.953933276,224.55501690000003,rice
21 88,35,40,23.57943626,83.58760316,5.85393208,291.2986618000001,rice
22 89,45,36,21.32504158,80.47476396,6.442475375,185.4974732,rice
23 76,40,43,25.15745531,83.11713476,5.070175667,231.3843163,rice
24 67,59,41,21.94766735,80.97384195,6.012632591,213.3560921,rice
25 83,41,43,21.0525355,82.67839517,6.254028451,233.1075816,rice
26 98,47,37,23.48381344,81.33265073,7.375482851,224.0581164,rice
```

Story 2: Import the required Python libraries for data analysis, visualization, and machine learning.

Necessary Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn are imported. These libraries are used for data handling, numerical operations, visualization, and analysis throughout the project.

```
[1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans

from sklearn.metrics import accuracy_score
```

Story 3: Read and explore the dataset to understand its structure, features, and target variables.

The dataset is loaded using Pandas and basic inspection methods like `head()`, `info()`, and `describe()` are used. This helps understand the structure, data types, feature columns, and target variable.

```
[2]: df = pd.read_csv(r"C:\Users\harsh\OneDrive\Desktop\OptiCrop\data\Crop_recommendation.csv")
print("Dataset Shape:", df.shape)
```

```
df.head()
```

```
Dataset Shape: (2200, 8)
```

```
[2]:
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

```
[3]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0    N            2200 non-null    int64
1    P            2200 non-null    int64
2    K            2200 non-null    int64
3    temperature  2200 non-null    float64
4    humidity     2200 non-null    float64
5    ph           2200 non-null    float64
6    rainfall     2200 non-null    float64
7    label        2200 non-null    str
dtypes: float64(4), int64(3), str(1)
memory usage: 153.0 KB
```

```
[4]: df.describe()
```

```
[4]:
```

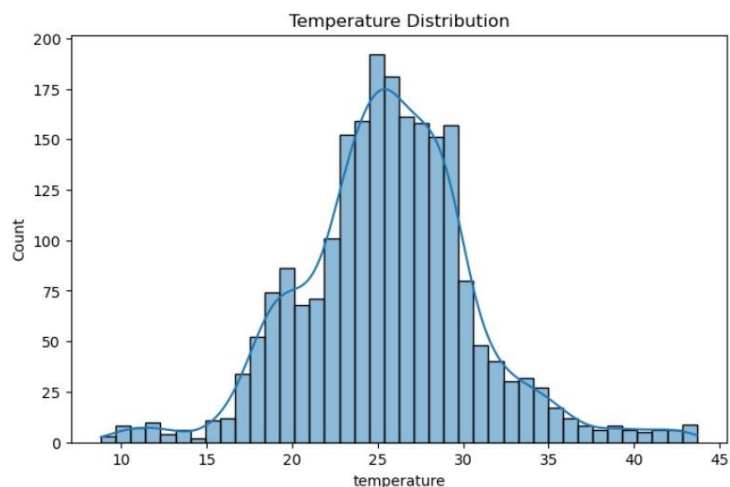
	N	P	K	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

Story 4: Perform univariate analysis to examine the distribution and characteristics of individual features.

Each feature in the dataset is analyzed individually to understand its distribution and range. Histograms and distribution plots are used to observe patterns in soil and environmental parameters.

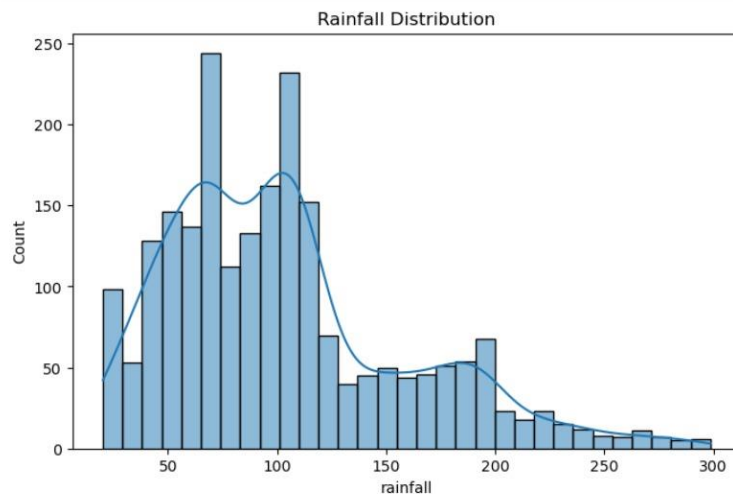
```
[7]: plt.figure(figsize=(8,5))
sns.histplot(df["temperature"], kde=True)

plt.title("Temperature Distribution")
plt.show()
```



```
[8]: plt.figure(figsize=(8,5))
sns.histplot(df["rainfall"], kde=True)

plt.title("Rainfall Distribution")
plt.show()
```



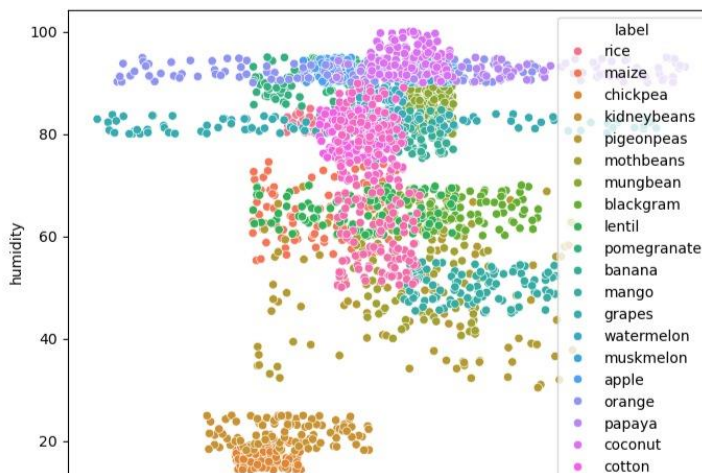
Story 5: Conduct bivariate analysis to identify relationships between agricultural parameters and crop suitability.

Relationships between two variables are analyzed, especially between input features and crop labels. This helps identify how individual parameters influence crop selection.

```
[9]: plt.figure(figsize=(8,6))

sns.scatterplot(
    x="temperature",
    y="humidity",
    data=df,
    hue="label"
)

plt.show()
```



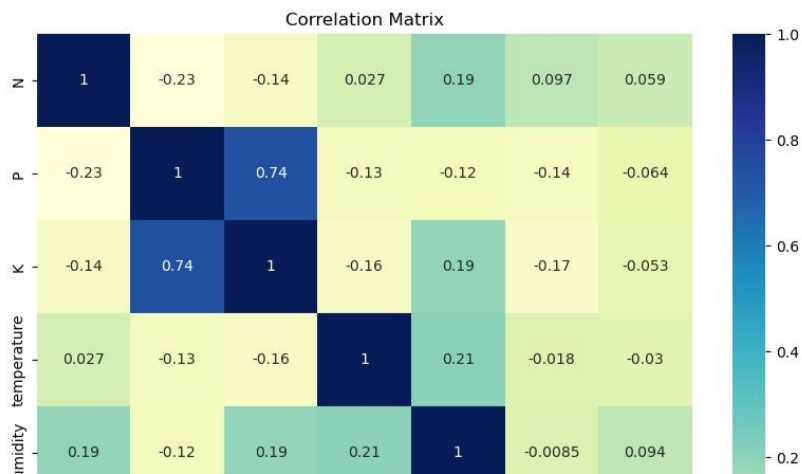
Story 6: Perform multivariate analysis to discover patterns and interactions among multiple variables.

Multiple features are analyzed together to understand combined effects on crop prediction. Heatmaps and correlation analysis are used to identify strong relationships between variables.

```
[10]: plt.figure(figsize=(10,8))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="YlGnBu"
)

plt.title("Correlation Matrix")
plt.show()
```



Epic 3: Data Pre-Processing

Story 1: Check for null values and handle missing data to improve dataset quality.

The dataset is checked for missing or null values using Pandas functions. Any missing data is handled appropriately to ensure data quality and prevent errors during model training.

```
[5]: df.isnull().sum()

[5]: N      0
     P      0
     K      0
     temperature  0
     humidity    0
     ph          0
     rainfall    0
     label       0
     dtype: int64
```

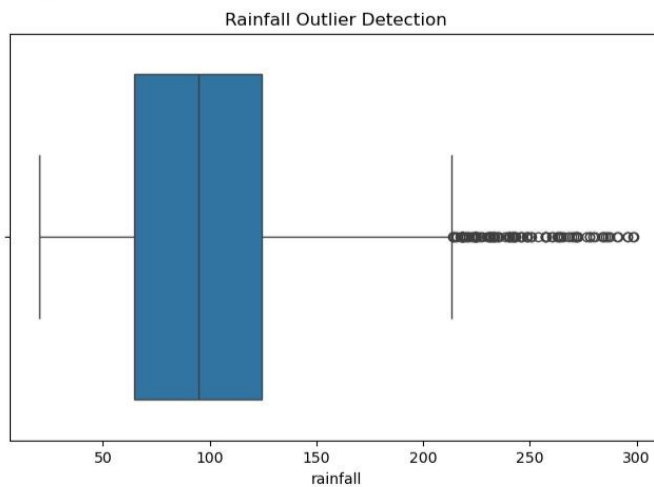
Story 2: Detect and treat outliers that may affect model performance.

Outliers are identified using statistical and visualization techniques such as box plots. Extreme values that may negatively affect model performance are analyzed and treated where necessary.

```
[11]: plt.figure(figsize=(8,5))

sns.boxplot(x=df["rainfall"])

plt.title("Rainfall Outlier Detection")
plt.show()
```



Story 3: Extract seasonal crop information and prepare relevant features for analysis.

Crop data is examined to identify seasonal patterns and classifications. Relevant seasonal information is extracted and prepared as additional features to support analysis and improve recommendations.

```
[12]: seasonal_crops = {
    "rice": "Kharif",
    "maize": "Kharif",
    "chickpea": "Rabi",
    "kidneybeans": "Kharif",
    "pigeonpeas": "Kharif",
    "mothbeans": "Kharif",
    "mungbean": "Kharif",
    "blackgram": "Kharif",
    "lentil": "Rabi",
    "cotton": "Kharif",
    "jute": "Kharif"
}

seasonal_crops
```

```
[12]: {'rice': 'Kharif',
      'maize': 'Kharif',
      'chickpea': 'Rabi',
      'kidneybeans': 'Kharif',
      'pigeonpeas': 'Kharif',
      'mothbeans': 'Kharif',
      'mungbean': 'Kharif',
      'blackgram': 'Kharif',
      'lentil': 'Rabi',
      'cotton': 'Kharif',
      'jute': 'Kharif'}
```

Story 4: Split the dataset into training and testing sets for model development and evaluation.

The dataset is divided into training and testing sets using Scikit-learn. The training set is used to build machine learning models, while the testing set is used to evaluate model performance and accuracy.

```
[13] X = df.drop("label", axis=1)

y = df["label"]

[14] encoder = LabelEncoder()

y = encoder.fit_transform(y)

[15] X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)

... (1760, 7)
(440, 7)
```

Epic 4: Model Building

Story 1: Apply K-Means Clustering to identify patterns and group similar agricultural conditions.

K-Means Clustering is applied to group similar agricultural conditions based on soil and environmental parameters. This helps identify hidden patterns in the dataset and provides insights into crop suitability under different conditions.

```
kmeans = KMeans(
    n_clusters=22,
    random_state=42,
    n_init=10
)

kmeans.fit(X)

print("K-Means Completed")

K-Means Completed
```

Story 2: Train a Logistic Regression model to predict suitable crops based on environmental parameters.

A Logistic Regression model is trained using the processed dataset. The model learns the relationship between environmental factors and crop types, enabling it to predict suitable crops for given input conditions.

```
lr_model = LogisticRegression(max_iter=5000)

lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)

lr_accuracy = accuracy_score(y_test, lr_pred)

print("Logistic Regression Accuracy:", lr_accuracy)
```

Logistic Regression Accuracy: 0.975

Story 3: Evaluate model performance using appropriate metrics and compare results.

The trained model is evaluated using performance metrics such as accuracy and classification reports. The results are analyzed and compared with other machine learning models to determine the most effective approach for crop prediction.

```
    knn ,
    "Logistic Regression",
    "Decision Tree",
    "Random Forest"
],
"Accuracy": [
    knn_accuracy,
    lr_accuracy,
    dt_accuracy,
    rf_accuracy
]
})
```

```
results.sort_values(
    by="Accuracy",
    ascending=False
)
```

[21]

	Model	Accuracy
3	Random Forest	0.993182
2	Decision Tree	0.986364
1	Logistic Regression	0.975000
0	KNN	0.970455

Story 4: Save the best-performing model for deployment and generate crop predictions based on user-provided inputs.

The model with the best performance is selected and saved using Pickle (.pkl) for future use. During prediction, user-provided inputs are processed and passed to the saved model to generate crop recommendations through the application.

```
import pickle
import os

os.makedirs("../models", exist_ok=True)

pickle.dump(
    rf,
    open("../models/crop_model.pkl", "wb")
)

pickle.dump(
    encoder,
    open("../models/label_encoder.pkl", "wb")
)

print("Model Saved Successfully")
```

Model Saved Successfully

Epic 5: Application Building

Story 1: Design and develop HTML pages to create an interactive and user-friendly interface.

The user interface is developed using HTML, CSS, and Bootstrap to create a simple and responsive web application. Input forms are designed to collect soil and environmental parameters, while result pages display the recommended crop in a clear and user-friendly format.

```

OptiCrop.py U  index.html X
C:\Users\harsh> OneDrive\ Desktop\ OptiCrop\ templates\ index.html >...
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5
6 <meta charset="UTF-8">
7 <meta name="viewport" content="width=device-width, initial-scale=1">
8
9 <title>OptiCrop</title>
10
11 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
12
13 <style>
14
15 body{
16     background:linear-gradient(135deg, #4CAF50, #8BC34A, #CDDC39);
17     min-height:100vh;
18     font-family:Arial,sans-serif;
19 }
20
21 .hero{
22     text-align:center;
23     color:white;
24     padding:30px 0;
25 }
26
  
```

Story 2: Build the Python backend code and integrate it with the trained machine learning model.

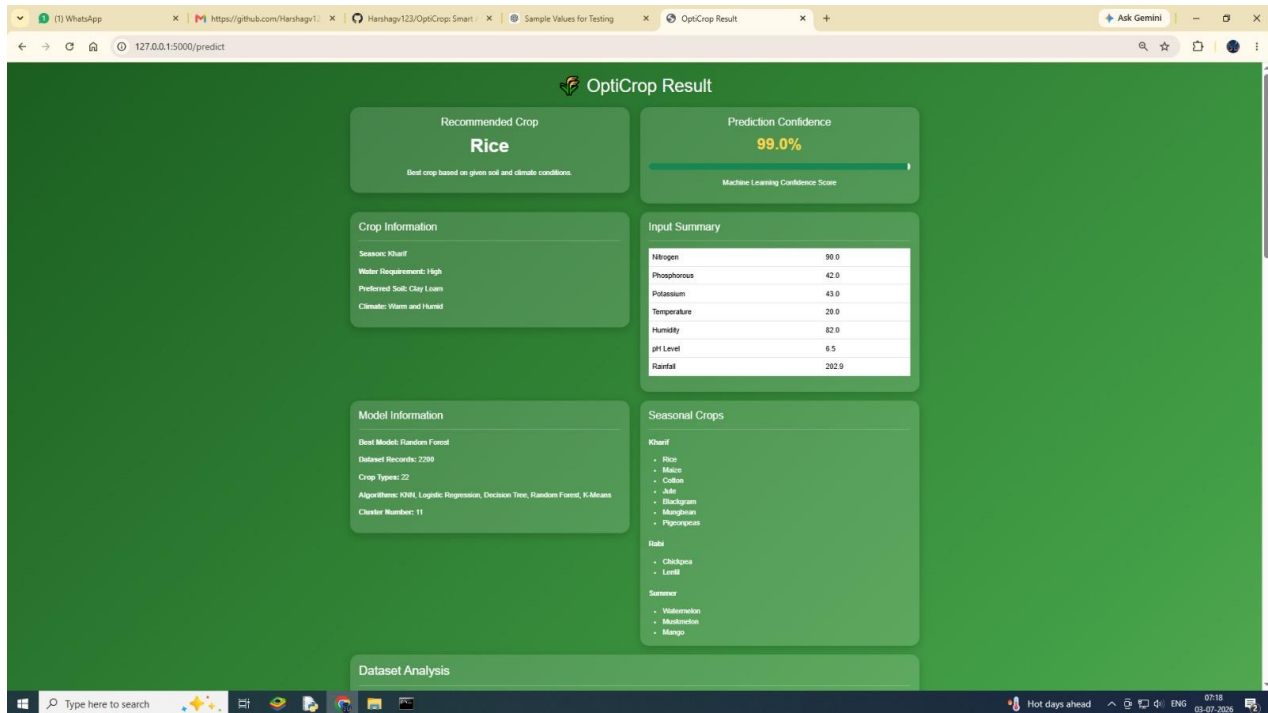
The backend is developed using Flask and Python. User inputs are received through web forms, processed, and passed to the trained machine learning model. The model generates crop recommendations, and the results are returned to the frontend for display. It uses app.py

```

Explorer (Ctrl+Shift+E)  index.html  result.html 4  app.py X
C:\Users\harsh> OneDrive\ Desktop\ OptiCrop\ app.py
1 from flask import Flask, render_template, request
2 import pickle
3 import numpy as np
4
5 app = Flask(__name__)
6
7 # Load Models
8
9 model = pickle.load(
10     open("models/crop_model.pkl", "rb")
11 )
12
13 encoder = pickle.load(
14     open("models/label_encoder.pkl", "rb")
15 )
16
17 kmeans = pickle.load(
18     open("models/kmeans_model.pkl", "rb")
19 )
20
21 try:
22     with open("models/model_name.txt", "r") as f:
23         model_name = f.read().strip()
24 except:
25     model_name = "Random Forest"
26
  
```

Story 3: Run, test, and validate the application to ensure accurate crop recommendations and smooth functionality.

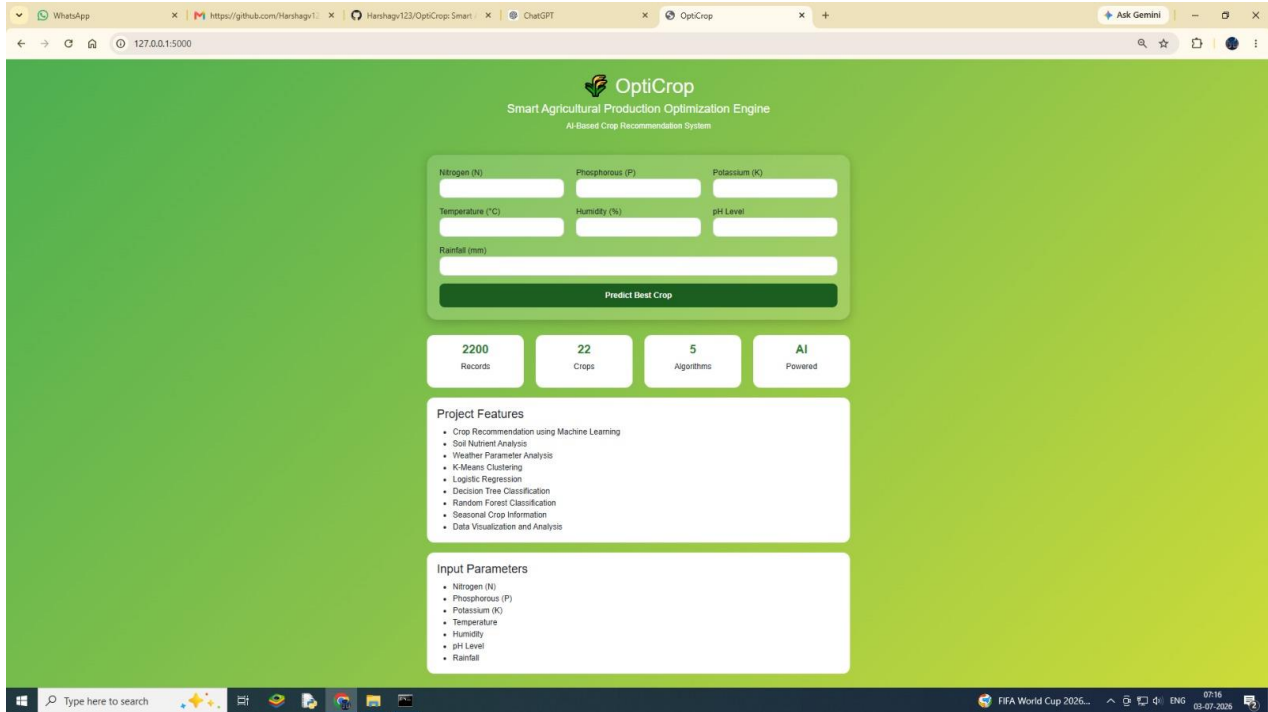
The application is executed using the Flask development server and tested with different input values. Model predictions, input validation, and page navigation are verified to ensure accurate crop recommendations and smooth system functionality.



Exploring the Website's Web Pages:

Home Page:

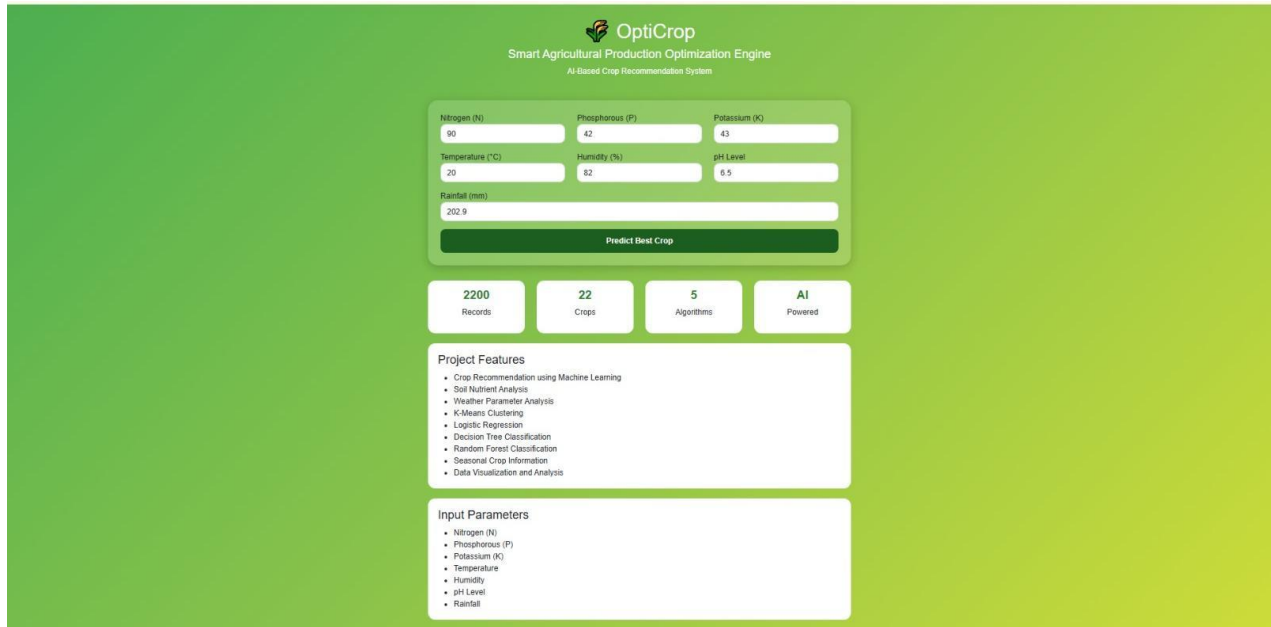
Description: The Home Page serves as the entry point of the OptiCrop application. It introduces users to the Smart Agricultural Production Optimization Engine and provides an overview of the system's purpose. The page contains navigation options and directs users to the crop recommendation feature. The design is simple, responsive, and user-friendly to ensure easy accessibility for farmers and other users.



The screenshot shows the OptiCrop web application interface. At the top, there's a header with the OptiCrop logo and the title "Smart Agricultural Production Optimization Engine". Below this, there's a form for inputting agricultural and environmental parameters. The form includes input fields for Nitrogen (N), Phosphorous (P), Potassium (K), Temperature (°C), Humidity (%), pH Level, and Rainfall (mm). A "Predict Best Crop" button is located below the input fields. Below the form, there's a section for project statistics: 2200 Records, 22 Crops, 5 Algorithms, and AI Powered. This is followed by a "Project Features" section listing various machine learning and data analysis techniques. Finally, there's an "Input Parameters" section listing the parameters used in the model: Nitrogen (N), Phosphorous (P), Potassium (K), Temperature, Humidity, pH Level, and Rainfall.

Input Section:

Description: The Input Page allows users to enter agricultural and environmental parameters required for crop prediction. Users provide values such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall through a structured form. Input validation is performed to ensure valid data is submitted before processing. Once the details are entered, users can click the prediction button to generate recommendations.



OptiCrop
 Smart Agricultural Production Optimization Engine
 AI Based Crop Recommendation System

Nitrogen (N): 90
 Phosphorous (P): 42
 Potassium (K): 43
 Temperature (°C): 20
 Humidity (%): 82
 pH Level: 6.5
 Rainfall (mm): 202.9

Predict Best Crop

2200 Records
 22 Crops
 5 Algorithms
 AI Powered

Project Features

- Crop Recommendation using Machine Learning
- Soil Nutrient Analysis
- Weather Parameter Analysis
- K-Means Clustering
- Logistic Regression
- Decision Tree Classification
- Random Forest Classification
- Seasonal Crop Information
- Data Visualization and Analysis

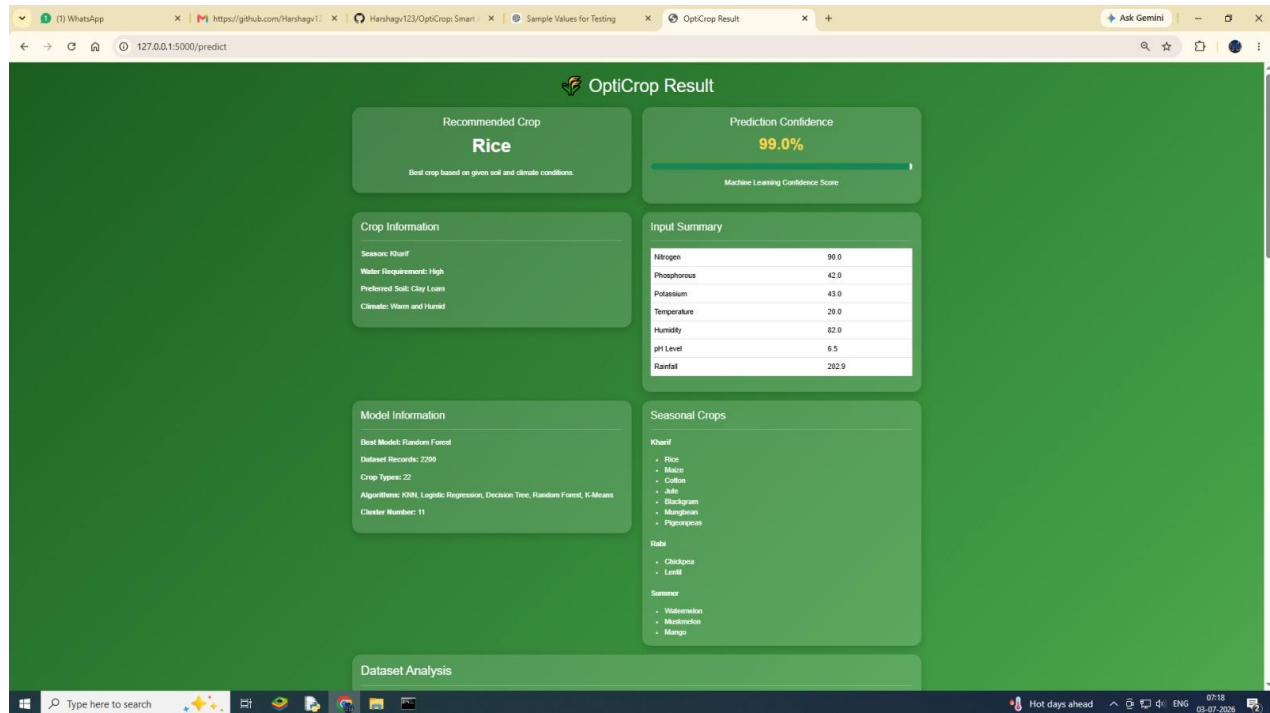
Input Parameters

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH Level
- Rainfall

Results Section:

Description:

The Result Page displays the crop recommendation generated by the machine learning model based on the user inputs. The predicted crop is presented clearly along with relevant information to help users make informed farming decisions. The page dynamically retrieves prediction results from the Flask backend and presents them in an easy-to-understand format. Users can also navigate back to the input page to perform additional predictions.



Conclusion:

OptiCrop is a machine learning-based agricultural recommendation system developed using Flask and various data science technologies to support intelligent farming decisions. The application analyzes important soil and environmental parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH level, and rainfall to recommend the most suitable crop for cultivation. By integrating data preprocessing, exploratory data analysis, machine learning model development, and web application deployment, OptiCrop provides a practical solution for improving agricultural productivity and resource utilization.

The project demonstrates how machine learning can be effectively applied to agriculture to assist farmers, researchers, and policymakers in making data-driven decisions. Through model training, evaluation, and deployment, the system delivers accurate crop recommendations using a simple and user-friendly interface. Looking ahead, OptiCrop can be further enhanced by integrating real-time weather data, fertilizer recommendation modules, yield prediction capabilities, and mobile application support. These improvements can transform the system into a comprehensive smart agriculture platform that promotes sustainable farming and efficient crop management.